

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

В.А. Миклуш
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Словарно-ориентированные методы
кодирования

по курсу: Теория информации, данные,
знания

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание и исходные данные	3
3 Ход выполнения задания	4
4 Ручная трассировка и сравнение результата.....	6
5 Вывод.....	8
6 Листинг с кодом программы	9

1 Цель работы

Целью данной лабораторной работы является изучить словарно-ориентированный метод кодирования LZ77, его структуру и принципы работы, освоить формирование кодов вида (offset, length, nextSymbol) и выполнить сравнение ручной трассировки с результатами программной реализации.

2 Задание и исходные данные

В соответствии с вариантом, таблица 1, построить алгоритм (блок-схему) и написать программу, реализующую заданный метод кодирования.

Таблица 1 – Исходные данные варианта 1

№ п/п	Метод кодирования	Текст
1.	LZ77	Интервьюер интервента интервьюировал

Провести ручную трассировку и сравнить полученные результаты между собой. Рассчитать среднее число элементарных сигналов.

3 Ход выполнения задания

Ход выполнения задания заключался в последовательной реализации всех этапов, предусмотренных методическими указаниями. Вначале на основе теоретических сведений об алгоритме LZ77 была построена блок-схема, отражающая логику работы метода. В блок-схеме последовательно показаны операции: выделение словаря и буфера в пределах скользящего окна, поиск максимального совпадения между началом буфера и подстрокой словаря, формирование тройки параметров смещения, длины совпадения и следующего символа, а также переход к дальнейшему фрагменту строки. Построенная схема представлена на рисунке 1 и используется как основа для программной реализации.

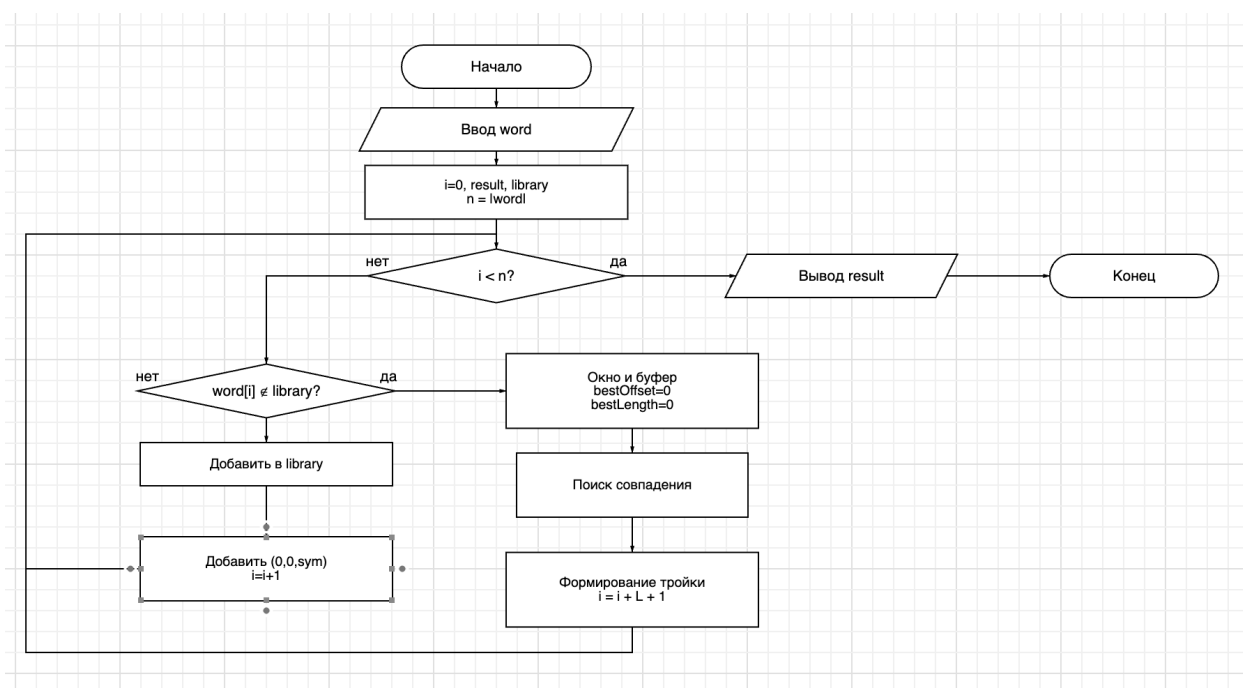


Рисунок 1 – Блок схема алгоритма LZ77

После создания блок-схемы была разработана программа, реализующая алгоритм кодирования LZ77 для строки «Интервьюер интервента интервьюировал». Программа последовательно просматривает входной текст, формируя словарь из уже обработанных символов и буфер из оставшейся части строки. Для каждого шага определяется максимальная длина совпадения

между содержимым буфера и подстрокой, найденной в словаре. На основе этих данных формируется выходная тройка вида (offset, length, next), соответствующая правилам алгоритма LZ77. После формирования тройки окно сдвигается вперёд на величину, равную длине совпадения плюс один символ.

Далее программа была запущена для обработки исходного текста, и в результате была получена последовательность троек кодирования. Затем выполнена ручная трассировка алгоритма, в ходе которой последовательно определялись состояния словаря и буфера, проводился поиск совпадений и формировались соответствующие тройки. Полученные вручную результаты были сопоставлены с программным выводом. Все тройки совпали, что подтверждает корректность разработанной программы и правильность построенной блок-схемы.

После этого на основе полученной последовательности кодов было произведено вычисление среднего количества элементарных сигналов согласно методике, описанной в методических материалах. Листинг программы представлен в приложении А.

4 Ручная трассировка и сравнение результата

В ходе выполнения четвертого пункта лабораторной работы была проведена ручная трассировка алгоритма LZ77 для строки «интервьюер интервента интервьюировал». Ручной анализ последовательности полностью совпал с программной реализацией, что подтверждает корректность работы алгоритма. На каждом шаге определялась максимальная совпадающая подстрока между окном и буфером, после чего формировались тройки вида смещение, длина, следующий символ. Всего было получено 16 троек, полностью совпадающих с результатами программы, то есть расхождений между ручной и автоматической трассировкой не выявлено. В таблице 2 можно наглядно увидеть результат ручной трассировки.

Таблица 2 – Результат ручной трассировки программы

Шаг	Окно	Буфер	Результат
1	-	интервьюер...	(0,0,и)
2	и	нтервьюер...	(0,0,н)
3	ин	тервьюер...	(0,0,т)
4	инт	ервьюер...	(0,0,е)
5	инте	рвьюер...	(0,0,р)
6	интер	вьюер...	(0,0,в)
7	интерв	ьюер...	(0,0,ь)
8	интервь	юер...	(0,0,ю)
9	интервью	ер интервента...	(4,2,пробел)
10	интервьюер	интервента...	(1,6,е)
11	интервьюер интерв	ента интервьюировал...	(2,2,а)
12	-	-	(0,0,пробел)
13	-	интервента...	(1,8,и)
14	-	...	(5,1,о)
15	-	...	(6,1,а)
16	-	л	(0,0,л)

После этого было рассчитано среднее число элементарных сигналов. Так как длина исходной строки составляет 36 символов, для полей offset и length

требуется по 6 бит ($\log_2 36$ округляется вверх). Алфавит содержит 12 уникальных символов, что требует 4 бит для поля next. Таким образом, одно кодовое слово содержит $6 + 6 + 4 = 16$ бит. Общее количество бит равно $16 \times 16 = 256$ бит. При делении на длину исходной строки $256 \div 36$ получается примерно 7,11 бит на символ. Это значение характеризует средний объем кодирования и подтверждает эффективность работы алгоритма в данных условиях.

5 Вывод

В ходе выполнения работы был реализован алгоритм LZ77 и проведена его ручная трассировка для строки «Интервьюер интервента интервьюировал». Результаты ручного и программного выполнения полностью совпали, что подтверждает корректность построенной блок-схемы и правильность реализации алгоритма. Полученная последовательность троек отражает структуру повторов в тексте и демонстрирует принципы работы скользящего окна.

Эффективность сжатия по алгоритму LZ77 зависит от выбранных размеров словаря и буфера, поскольку от них определяется длина двоичных кодов смещения и длины совпадения. При оптимальном выборе параметров кодирование может давать выигрыш по сравнению с исходным представлением текста, однако при больших значениях словаря часть преимущества теряется из-за увеличения служебной информации. В целом работа позволила на практике подтвердить особенности и ограничения метода LZ77.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
let word: string = "Интервьюер интервента интервьюировал".toLowerCase();
```

```
function moveWindowLZ77(word: string): Array<{ offset: number, length: number, next: string }> {
```

```
    const result: Array<{ offset: number, length: number, next: string }> = [];
```

```
    let wordLength: number = word.length;
```

```
    let uniqueSymbols: number = new Set(word).size;
```

```
    let library: string[] = [];
```

```
    console.log(wordLength, uniqueSymbols);
```

```
    let i = 0;
```

```
    while (i < wordLength) {
```

```
        if (!library.includes(word[i])) {
```

```
            library.push(word[i]);
```

```
            console.log("запушил ", word[i]);
```

```
            result.push({ offset: 0, length: 0, next: word[i] })
```

```
            i += 1;
```

```
        } else {
```

```
            const window = word.slice(0, i);
```

```
            const buffer = word.slice(i);
```

```
            let bestOffset = 0;
```

```
            let bestLength = 0;
```

```

    for (let len = 1; len <= buffer.length; len++) {
        const sub = buffer.slice(0, len);
        const idx = window.indexOf(sub);
        if (idx !== -1) {
            bestOffset = idx + 1; // offset от начала строки, индекс с 1
            bestLength = len;
        } else {
            break;
        }
    }

    const nextSymbol = (i + bestLength) < wordLength ? word[i +
bestLength] : "";

    result.push({
        offset: bestOffset,
        length: bestLength,
        next: nextSymbol
    });

    console.log(
        `словарь=[${window.slice(-9)}] буфер=[${buffer.slice(0, 7)}] ->
        (${bestOffset}, ${bestLength}, ${nextSymbol || "Ø"})`
    );
    i += bestLength + 1;
}
}

return result;
}

```

```
console.log(moveWindowLZ77(word));
```